FretAid — Project Report

An interactive guitar and bass fretboard, scale, chord and tuner application

Build report
6 October 2026

1. Summary
2. What was built
   - 2.0 Revisions
   - 2.1 The instrument view
   - 2.2 The four interaction modes
   - 2.3 Clicking a position
3. Architecture
   - 3.1 State versus derived data
   - 3.2 Data-driven instruments
   - 3.3 Rendering performance
4. The music-theory engine
   - 4.1 Pitch is an integer
   - 4.2 The fretboard
   - 4.3 Enharmonic spelling, systematically
   - 4.4 Chords and scales kept distinct
5. Alternate tunings and the chord engine
   - 5.1 There are no stored chord shapes
   - 5.2 Power chords
   - 5.3 Custom tunings
6. The audio engine
   - 6.1 Why Karplus–Strong, and why rendered offline
   - 6.2 Strumming
7. The tuner
   - 7.1 Signal path
   - 7.2 Measured accuracy
   - 7.3 The camera move
8. Interface, accessibility and resilience
   - 8.1 Visual design
   - 8.2 Responsive design
   - 8.3 Accessibility

8.4 Error handling
8.5 Settings that persist
8.6 Six languages

9. Testing results
9.1 Checks run in a real browser
9.2 Items from the brief’s development checklist

10. Running and deploying
10.1 Locally
10.2 Free hosting, no domain required
10.3 Microphone permission

11. What is supported
11.1 Instruments
11.2 Tuning presets
11.3 Scales and modes (27)
11.4 Chords (21)
11.5 Languages

12. Honest limitations
12.1 Natural next steps

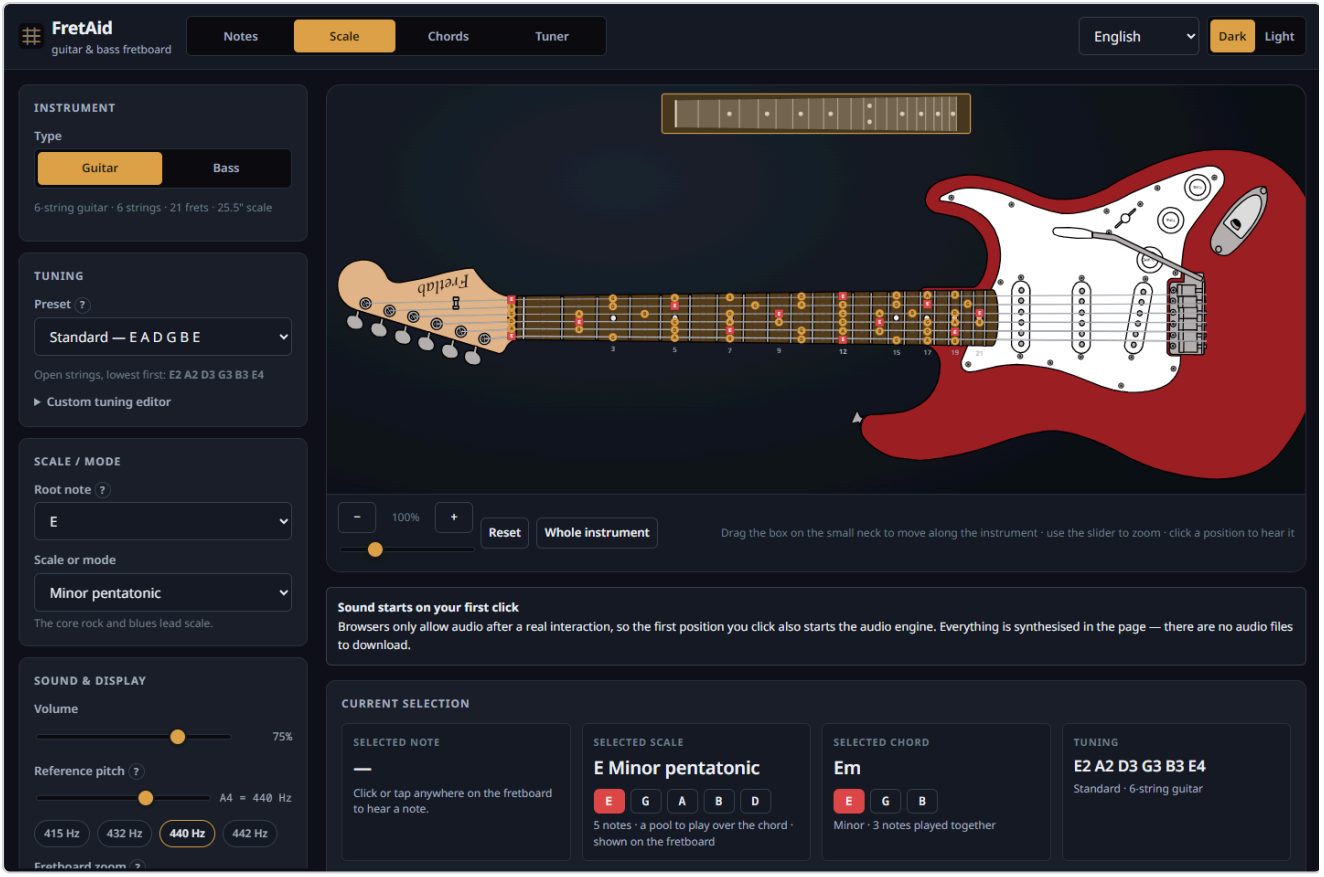
13. Deliverables

1. Summary

FretAid is a complete, working web application for guitar and bass learners. It is not a prototype: every button described below does the thing it says, and the project is ready to run locally and to deploy to free hosting without buying a domain.

Stack	React 18, TypeScript 5.6, Vite 5, Zustand 4
Instruments	6-string guitar, 4-string bass
Tunings	20 presets plus a custom tuning editor per instrument
Scales	27 scales and modes
Chords	21 chord types, including two kinds of power chord
Audio	Karplus–Strong plucked-string synthesis, Web Audio API, no samples
Tuner	Microphone pitch detection, accurate to about one cent
Languages	English, 한국어, 日本語, 简体中文, 繁體中文, Español
Tests	280, all passing
Production bundle	393 kB JavaScript (131 kB gzipped), 16 kB CSS
Deployment	GitHub Pages workflow included; Cloudflare, Netlify, Vercel configs included
Cost to run	Nothing. No backend, no account, no domain.

The single design decision that the rest of the project follows from: **the music-theory engine owns all musical truth, and it is derived from the current tuning every time.** There is no stored table of fret positions and no stored table of chord shapes anywhere in the codebase. That is what makes alternate tunings correct rather than approximately correct.



The default view: a 6-string guitar in standard tuning with E minor pentatonic highlighted across the neck.

---

2. What was built

2.0 Revisions

Revision 8

- **The application is now called FretAid.** Another site already uses the earlier name. Every mention in the interface, in all six languages, in the code and in the documentation was changed. Three things deliberately were not: the logo inside the instrument drawings, which is the owner's to redraw; the project folder; and the GitHub repository, whose address the live site is served from. Settings saved under the old name are still read, so nobody's instrument, tuning or language is reset by the rename.
- **Zooming no longer makes the instrument section taller.** The section keeps one height; zooming in enlarges the instrument inside it and crops what does not fit, with the neck's centre line held on the middle of the section at every zoom.
- **The box on the small neck no longer snaps to the ends.** A drag moves the view by exactly the distance the pointer travels, all the way to either end of the instrument.

Revision 7

- **Moving along the neck is done on a small map of the neck** above the stage: the nut, all 21 frets and the position dots, and nothing else, drawn to the neck's real proportions and taper. A semi-transparent box marks the part being shown; dragging it, pressing the map, or the arrow keys move the view. The map is drawn from the same geometry as the instrument, so the box covers exactly the frets on show.
- **The instrument drawings are levelled.** The supplied drawings were each slightly tilted, so the neck ran downhill: by 0.63° on the guitar, about seven units from nut to body, and by 0.09° on the bass. Every SVG in each instrument's folder — the whole drawing and each part — has been turned by that instrument's own angle, by wrapping its contents in a single rotation rather than rewriting them, so the parts still fit together exactly. The necks now measure level to a thousandth of a degree.
- **The headstock logo is as drawn.** An earlier version turned it the right way up; it is upside down in the drawings on purpose, and is left that way.
- **The wheel no longer zooms or pans.** Scrolling over the instrument scrolls the page up and down and does nothing else. Zooming is the − and + buttons and a slider directly beneath them, and a zoom now keeps the middle of the view in the middle instead of sliding the neck sideways.
- **On a narrow screen the guitar / bass switch is at the top of the page**, above the instrument it changes, rather than in a card below it.
- **Every piece of text is collected in** `docs/texts.md`, organised by where it appears, with all six languages side by side and the placeholders named. It is generated from the catalogues by `npm run texts`.

Revision 6

- **The tuner now hears a string played softly.** It was going deaf long before the detector did: a fixed level gate discarded signals the pitch detector still read perfectly, so a gentle pluck, or a microphone across the room, registered as silence. The gate now follows the room instead of sitting at a fixed level, and it starts 34 dB lower. Measured on the same synthesised plucks: detection down to **1/50th of the level it previously needed**, with the pitch still accurate to a cent. Section 7.2 has the numbers.
- **A microphone with a DC offset no longer produces a confident wrong note.** An offset is level without being sound: it passed the old gate and then dominated the correlation. Measured before the fix: a reading **3780 cents** — more than three octaves — from the right answer, at 0.95 confidence.
- **The signal meter is in decibels**, so a quietly played string visibly moves it instead of leaving it pinned at nothing.

Revision 5

- **The whole interface is available in six languages**: English, Korean, Japanese, Simplified Chinese, Traditional Chinese and Spanish. Every visible string is translated, including the ones nobody sees — the accessible name of each of the 132 fretboard positions, the tuner's spoken readings, the crash screen and the help bubbles — and so are the scale, chord and tuning

catalogues, which are the part a learner actually reads. Section 8.6 explains how it is built and what it guarantees.

Revision 4

- The chord view shows the chord chooser alone; the scale chooser belongs to the scale view.
- **Strings are laid against the drawn fretboard's real edges.** The artwork's neck is not perfectly level and its centre line is not at y = 0 — the guitar's sits about eight units low — so assuming a horizontal neck put the strings off the board. The extractor now samples the drawn board's top and bottom edges along its length, and the strings follow them, continuing the same fan past the end of the board out to the bridge, where they now terminate at the string anchor rather than at the saddle line.
- The bass's 21st fret is playable; both instruments have 21.
- An out-of-tune tuning machine is ringed in a muted red rather than amber, which was hard to see against pale maple.
- **On Auto the tuner's meter measures against the nearest note of the chromatic scale**, so it is as useful on a fretted note as on an open string; pinned to a string it measures against that string, as before.
- **A movable barre shape is always offered when one exists.** Scoring favours open shapes, so barres were being crowded out of the list; one is now kept deliberately. Where a barre is impossible — a power chord or an augmented triad in a tuning in fourths can never put two chord tones on one fret — the panel says so instead of leaving it unexplained.

Revision 3

- **The instrument graphics are now built from supplied layered SVG drawings** of a Stratocaster-style guitar and a Precision-Bass-style bass, a file per part. `tools/extract-artwork.mjs` reassembles the parts into one coordinate system by matching shape sizes, fits the equal-tempered fret rule to the drawings' own fret lines to recover the nut and scale length (to better than one part in a thousand), and re-expresses everything with the nut at the origin and the scale length at 1000 units — so FretAid's computed notes land on the drawn frets. See `reference/README.md`.
- **Pitch detection was wrong on most strings**, which is what made the tuner's meter look as though it worked only for the lowest string. Three independent faults, all fixed and covered by a test over every open string of every tuning preset: an anti-alias filter whose corner sat near 290 Hz; a decimation factor derived from the search range alone, which left a bass's period only fifteen samples long; and a peak search that stepped over the true period on strongly periodic signals. 38 failures became none.
- **Pinning a string in the tuner is now obvious and reversible**, and when a different string is played against a pinned target the readout says so instead of leaving the needle pegged at one end.
- **The 7-string guitar and 5-string bass were removed.**
- **A reference-pitch setting** (A4, 392-466 Hz) retunes playback, the tuner's targets and every displayed frequency.

- The scale and chord choosers appear only in their own modes; fret numbers appear only on frets that carry position markers.

Revision 2

After the first build was reviewed, six changes were made:

1. **The body and headstock outlines were redrawn.** They are now traced outlines of an offset double-cutaway guitar and an offset bass — real horns, cutaway scoops, a pinched waist and bouts — rather than the smooth profiles of the first version, which read as featureless blobs.
2. **The instrument was turned the right way up**: the lowest-pitched string is now drawn at the bottom, as a chord chart or tab staff reads, and the body and headstock were re-authored to match.
3. **The fret wires became visible.** They were being painted with an SVG gradient in `objectBoundingBox` units; a perfectly vertical line has zero bounding-box width, so the gradient degenerated and nothing was drawn. They are now solid metal with a bright crown.
4. **Clicking the selected position now deselects it.**
5. **The missing middle string was fixed** — the same gradient fault. A perfectly horizontal line has zero bounding-box height, and only the exact middle string of an odd-numbered set is perfectly horizontal, which is why it was the 4th string of 7 and the 3rd of 5 that vanished. One fix covered both this and the frets.
6. **The instrument selector became two steps**: Guitar or Bass, then the string count, with the instrument last used in each family remembered.

2.1 The instrument view

The instrument is drawn as a single SVG in one coordinate system: headstock, nut, fretboard, body, hardware and strings. The body and headstock are traced outlines held as data, so the silhouette can be adjusted without touching any rendering code. The lowest-pitched string is drawn at the bottom. The strings are drawn as one continuous run from each tuning peg, over the nut, to the bridge, which is what makes the fretboard and the instrument graphic read as one object rather than two unrelated pictures. Fret spacing uses the real equal-tempered rule (`1 − 2^(−fret/12)`), so the neck narrows the way a neck does and the 12th fret falls exactly halfway to the bridge.

The view is moved with a small map of the neck above it: a box marks the part on show and is dragged along the neck. It zooms with buttons, a slider beneath them and pinch gestures on a touch screen, and has a “Whole instrument” control that zooms out to show the complete guitar or bass including the body. The mouse wheel is left to the page, so it only ever scrolls up and down.



The "Whole instrument" view, showing the body, pickups, bridge and controls generated from the same geometry as the neck.

All of the artwork is generated by the project's own code from profile data in `components/fretboard/geometry.ts`. No commercial or copyrighted instrument imagery, samples or brand marks are used anywhere.

2.2 The four interaction modes

Notes — every position is clickable and plays the pitch it actually produces.

Scale — the selected scale is highlighted across the whole neck, with note names (or scale degrees) printed on each marker and the root distinguished.

Chords — calculated chord shapes, with playback and strumming.

Tuner — the view travels to the headstock and a chromatic tuner appears integrated with the tuning machines.

2.3 Clicking a position

Clicking or tapping anywhere in a fret space determines the note from the instrument, the string, the fret and the current tuning; displays its name, octave, frequency and MIDI number; and plays it. The hit area is the whole fret space rather than the dot, so it stays usable with a fingertip.

---

3. Architecture

The organising rule: **the music-theory engine knows nothing about React, and the user interface never does arithmetic on note names.**

```
src/
  core/                    pure TypeScript, no React, no DOM, unit-tested
    theory/pitch.ts        note names <-> MIDI <-> frequency <-> cents
    theory/spelling.ts     systematic enharmonic spelling
    theory/scales.ts       scale catalogue and pitch-class sets
    theory/chords.ts       chord catalogue and chord-to-scale relations
    theory/fretboard.ts    string + fret + tuning -> pitch
    theory/voicing.ts      tuning + chord -> playable fret shapes
    instruments/           instruments and tunings as data
    audio/pluck.ts         offline Karplus-Strong string synthesis
    audio/AudioEngine.ts   Web Audio graph, caching, strum scheduling
    tuner/pitchDetect.ts   NSDF pitch detection
    tuner/analysis.ts      frequency -> note, cents, target string
    tuner/TunerEngine.ts   microphone capture and the analysis loop

  state/store.ts           settings only, persisted to localStorage
  state/selectors.ts       settings -> derived musical context

  components/fretboard/    geometry, the instrument SVG, the stage/camera
  components/panels/       control, information, chord and tuner panels
  components/ui/           accessible form primitives
  hooks/                   React bindings for the engines
  tests/                   170 tests
```

3.1 State versus derived data

The store holds only *choices*: `"guitar6"`, `"drop-d"`, `"E"`, `"minor-pentatonic"`, a volume, a theme. Note names, highlighted positions and chord shapes are **derived** on demand in `selectors.ts` and memoised.

This matters more than it sounds. Because there is no second copy of the musical truth, it is not possible for the fretboard labels to disagree with the chord shapes, or for playback to disagree with the tuner. Changing one tuning preset necessarily updates all of them, because they are all computed from the same function's result.

3.2 Data-driven instruments

Adding an instrument means adding one object to `INSTRUMENTS`: name, family, string count, fret count, tuning presets, string gauges, scale length and a body style. Nothing in the renderer, the engines or the panels branches on instrument identity. An 8-string guitar, a 6-string bass or a ukulele would need no new rendering or engine code.

3.3 Rendering performance

The instrument is one SVG; each position is a memoised component. Playing a note re-renders one marker, not the neck. Scrolling and zooming are native scrolling and CSS transforms rather than React state churn, and the chord search — the one genuinely expensive derivation — is cached by tuning, root and chord.

---

4. The music-theory engine

4.1 Pitch is an integer

Everything internal is a MIDI note number: C4 = 60, A4 = 69. Note *names* exist only at the edge, for display, and are always derived. No part of the application manipulates note names as strings.

Function	Meaning
`noteNameToMidi("E2")` → `40`	name → pitch
`midiToNote(40)` → `E2`	pitch → name
`midiToFreq(40)` → `82.41 Hz`	pitch → frequency
`freqToNearestNote(438)` → `A4, −7.9 cents`	frequency → pitch and deviation
`midiAt(openMidis, string, fret)`	string + fret + tuning → pitch
`scalePitchClasses(root, scale)`	scale + root → pitch-class set
`chordPitchClasses(root, chord)`	chord + root → pitch-class set
`findVoicings(openMidis, root, chord)`	tuning + chord → playable shapes

4.2 The fretboard

A fret position is `openStringMidi + fretNumber`. That is the whole model. There is no table of fret positions, so there is nothing that could be right for standard tuning and wrong for anything else.

Scale highlighting works on **pitch class**: a position lights up when its pitch class belongs to the scale's set. It is never a standard-tuning pattern copied around. In Drop D, the lowest string's highlighted frets for E minor pentatonic become `0 2 5 7 9 12` instead of `0 3 5 7 10 12`, because the open string is now D — and the engine simply computes that rather than being told.

4.3 Enharmonic spelling, systematically

Spelling is the part of a fretboard application that usually degenerates into a list of exceptions. Here it is one rule.

Every scale and chord carries two parallel arrays: chromatic `intervals` (semitones from the root) and generic `degrees` (which letter the note is written on). The degree fixes the letter; the semitone count then fixes the accidental. That is exactly how notation works, and from it, with no special cases anywhere:

- B♭ major spells **Bb C D Eb F G A** — never A♯ C D D♯ F G A.
- F♯ major spells **F# G# A# B C# D# E#**, including the E♯ that beginners' tools get wrong.
- The A blues scale spells **A C D Eb E G**, with the ♭5 and the ♮5 written on the same letter, as the scale actually is.
- C diminished 7th spells **C Eb Gb Bbb**, with the double flat.

Notes outside the current key fall back to the key's own accidental side, so a fretboard in E♭ minor never shows a stray A♯ next to a B♭. Choosing a root of `Bb` rather than `A#` in the controls is how the player selects the key's spelling — the preference the brief asked for, implemented as a rule rather than as a list.

4.4 Chords and scales kept distinct

The brief was explicit that a learner must not confuse a chord voicing with a scale, and the interface is built around that. They are separate panels with separate note lists and separate explanations ("3 notes played together" versus "a pool to play over the chord"). The chord panel then lists the scales that fit the selected chord, each with a one-line reason, and **every suggestion is verified to contain every chord tone before it is offered** — so a dominant 7th is never paired with a plain major scale, which does not contain its flat 7th. Clicking a suggestion switches the fretboard to that scale and says so.

---

5. Alternate tunings and the chord engine

This was the most important correctness requirement in the brief, and it is where the project puts most of its effort.

5.1 There are no stored chord shapes

`findVoicings()` searches the fretboard. For each hand position along the neck it enumerates, per string, the frets whose **actual sounding pitch class** belongs to the chord, plus the open string and "muted". It prunes anything needing more than four fingers or a stretch wider than four frets, requires every non-optional chord tone to be present and the root to sound, then scores the survivors for playability: root in the bass, fewer fingers, lower on the neck, fuller voicing, no awkward dead strings in the middle.

The consequences are all verified by tests:

- **In standard tuning the search rediscovers the shapes players already know.** The open C (`x32010`), E minor (`022000`), A minor (`x02210`), D (`xx0232`), G (`320003`), E7 (`020100`), A7 (`x02020`),

Cmaj7 (`x32000`) and Am7 (`x02010`) all come out of the solver. None of them was typed in.

- **In D standard it does not offer `022000` as E minor.** In that tuning those frets sound a D minor chord, so the solver offers that shape under D minor, where it belongs. A familiar shape is never mislabelled as a chord it does not produce.
- **If a chord genuinely cannot be fingered in the current tuning**, the panel says so in plain language instead of showing something that would sound like a different chord.

Every shape is displayed together with the notes it actually sounds, computed from the live tuning, and with any chord tone it had to omit.



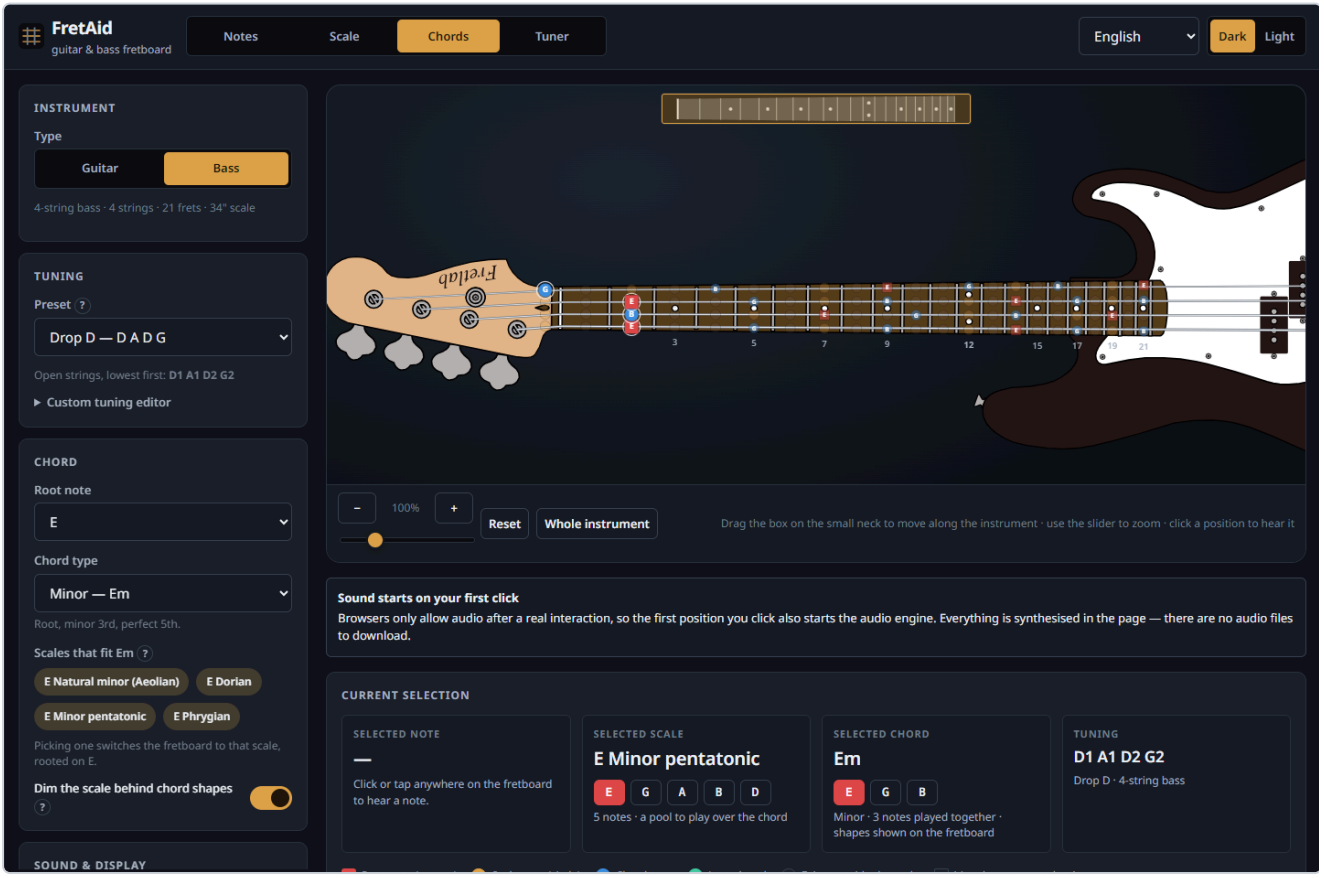
Chord mode: calculated Em shapes for the current tuning, each with its own diagram and fret string.

5.2 Power chords

Power chords are generated separately, because they are what most guitarists actually use alternate tunings for. The fifth and the octave are located **by pitch**, not by a fret offset. One piece of code therefore produces:

- the familiar two-fret offset in standard tuning;
- a **three-fret** offset when the root sits on the G string, because the interval to the B string is a major third rather than a fourth — the case hard-coded shape tables get wrong;
- the **one-finger, flat shape** in Drop D, where all three strings land on the same fret;
- correct shapes on the bass, whose four strings are all a fourth apart.

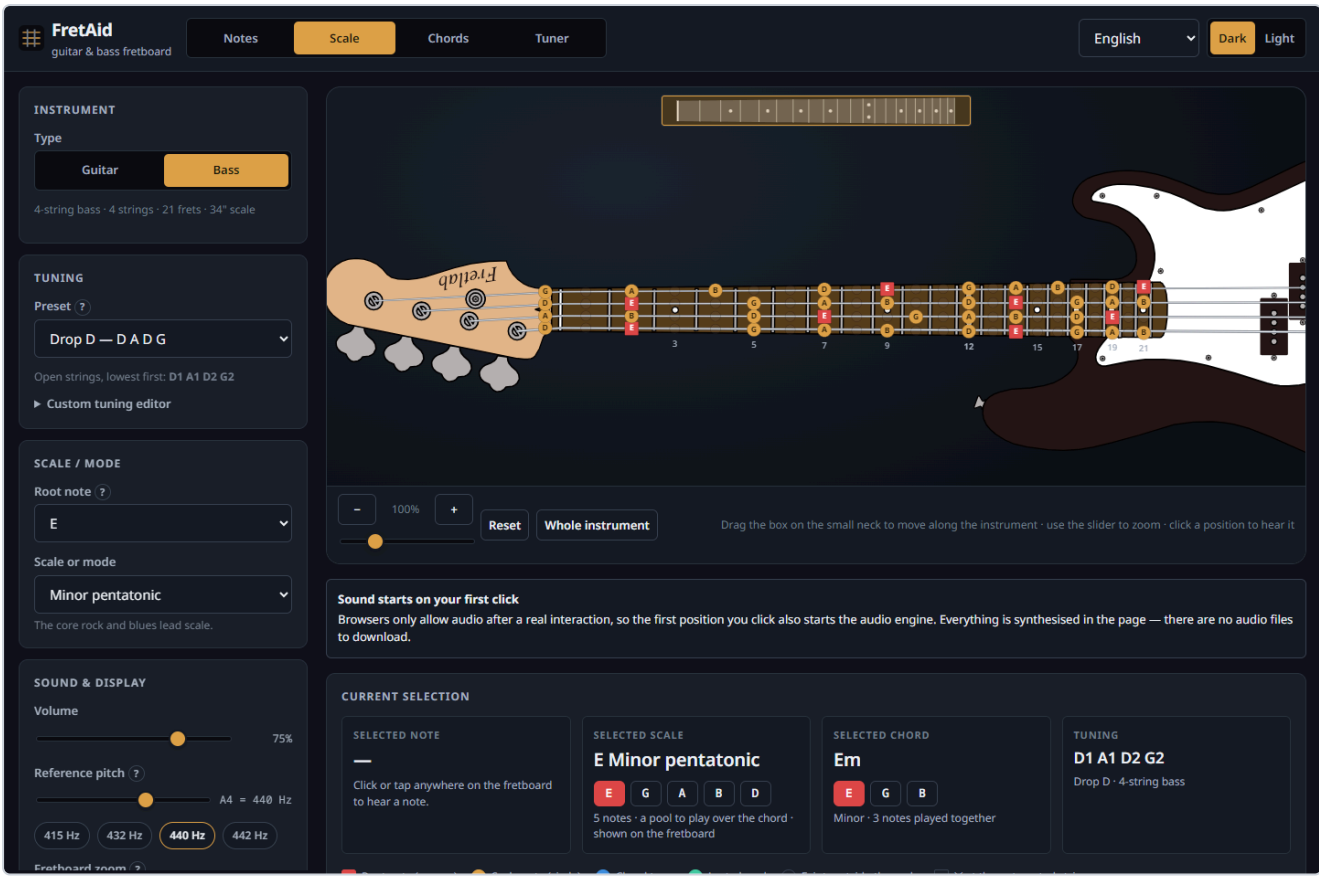
A test asserts that across seven instrument-and-tuning combinations and five roots, every generated power chord sounds exactly a root and a fifth (plus an octave when asked), with the root in the bass.



A bass in Drop D. The tuning, the note names, the highlighting and the chord shapes are all recalculated.

5.3 Custom tunings

Each instrument keeps its own custom tuning. The editor validates every string as it is typed, names the offending string when it is wrong, and refuses to apply an invalid tuning rather than letting it reach the fretboard. There are also one-click transpose-everything-by-a-semitone buttons.



A bass with a scale highlighted across the neck. The headstock layout and the string spacing change with the instrument.

---

6. The audio engine

Notes are **synthesised, not sampled**: there are no audio files to download and no licensing questions.

6.1 Why Karplus–Strong, and why rendered offline

The model is Karplus–Strong: a short burst of filtered noise circulates in a delay line one period long, losing a little energy and a little treble on every lap — which is what a plucked string does. A pick-position comb filter shapes the excitation, so the attack has the hollow character of a pluck rather than a uniform hiss.

The obvious implementation — a `DelayNode` in a Web Audio feedback loop — does not work. A delay inside a cycle is quantised to a 128-sample block, which caps usable pitch at roughly 344 Hz and detunes everything above it. So each note is instead rendered **offline into a sample buffer** in JavaScript, with a fractional delay line implemented by linear interpolation between two taps, and the loop filter's half-sample of delay accounted for explicitly.

The result is measurably in tune. A test renders every open string and a range of fretted notes and measures the fundamental of the rendered audio by autocorrelation: **every note is within 5 cents**,

from a 31 Hz B0 — below anything the shipped instruments reach, kept as the synthesiser's lower bound — to the 21st fret of a guitar's top E, at 44.1, 48 and 96 kHz.

Each note then passes a short EQ chain standing in for the instrument body — different for guitar and for bass — and a limiter, so a six-string strum does not clip. Buffers are cached per pitch, so repeated notes cost nothing.

6.2 Strumming

A chord is scheduled on the audio clock against a **single** reading of `currentTime`, so the sweep is exactly even; reading the clock once per note lets real elapsed time creep in and makes the strum uneven. Measured in a real browser, a six-string down strum at the normal preset schedules its strings at 0, 32, 64, 96, 128 and 160 ms.

- **Together** plays the strings with only a few milliseconds of humanising spread.
- **Down** runs from the lowest-pitched string upward.
- **Up** runs from the highest-pitched string downward.
- **Muted strings are skipped entirely** and make no sound.
- Speed is Slow (70 ms), Normal (32 ms), Fast (12 ms) or any custom value.

Thin strings are rendered brighter than thick ones, which is part of why a down strum and an up strum sound genuinely different rather than merely reversed.

---

7. The tuner

7.1 Signal path

1. **Capture.** `getUserMedia` is called **only** when tuner mode is entered, with echo cancellation, noise suppression and automatic gain control explicitly off — all three distort a sustained musical tone enough to move its detected pitch. Frames of 8192 samples (about 170 ms at 48 kHz) are read from an `AnalyserNode`, long enough to contain several periods of a 31 Hz low

   B.

2. **Conditioning.** The frame's mean is subtracted and anything below half the lowest note being searched for is rolled off. This is not cosmetic: a microphone with a DC bias — common on built-in and USB inputs — adds a constant that the level gate counts as signal, so a silent room reads as loud, and the constant then dominates the correlation. Before this was added, a 2% offset produced a confident reading three octaves from the truth. At the bass's lowest open string the roll-off costs under a decibel, and uniformly across the frame, so the correlation is untouched.

3. **Gating.** A frame is only analysed if it is louder than the room. The gate is not a fixed number: it sits a fixed distance (about 10 dB) above the **quietest** of the last four seconds, between a floor of −72 dBFS and a ceiling of −48 dBFS. Each of those choices answers a specific failure:

- *Why follow the room at all.* A fixed gate cannot suit both a quiet room and a noisy one. Set high enough for the noisy room — which is where it was — the tuner ignores a softly played string. Set low enough for the quiet room, a steady mains hum gets through, and hum is periodic, so the detector reports it as a confident note sitting between the bass's A and D strings.
- *Why the quietest frame and not the average.* An average is dragged up by the note being played, and the gate then rises and cuts the note off during its own decay. Measured, before this was a minimum: a bass note gated at 0.5 s while still four times louder than the floor. A note is a loud interval between quiet ones; a hum or a fan is in every frame, including the quietest.
- *Why a warm-up.* For the first four seconds the gate stays at its floor. Someone who opens the tuner and plays immediately would otherwise have the first frames of their own note taken for the room.
- *Why a ceiling.* The floor is the quietest recent frame, so a tone held for longer than the window would eventually become its own background and gate itself off. The ceiling sits well below any real playing level, so a sustained note stays audible indefinitely — verified over 20 s — and still above the hum the gate is there to reject.

The gate is deliberately *not* what rejects noise. That is the clarity threshold's job, and it does it well: pink room noise and white noise are rejected at every level tested, up to −35 dBFS, because neither is periodic.

4. **Detection.** The McLeod normalised square difference function, the normalised cousin of autocorrelation, is evaluated and the **first** strong peak is taken rather than the tallest. This is the detail that matters: a periodic signal correlates just as well at twice its period, and "tallest wins" is precisely what makes cheap tuners read an octave low on a bass string. The work is done in two stages — a decimated pass behind a four-pole anti-aliasing filter to find the period cheaply, then a refinement at the full sample rate with parabolic interpolation, which brings the estimate to about a cent. A median filter over the last five frames stops the needle twitching.

5. **Interpretation.** The frequency becomes a reading: the nearest chromatic note, and the cents deviation from the nearest string of **the tuning currently selected**. Drop C, E♭ standard and fully custom tunings therefore work with no special case. A string can be pinned instead of auto-selected, and clicking a string also plays its reference pitch.

6. **Display.** Note name, exact detected frequency, target frequency, signed cents, a needle, and the verdict written out in words — "Flat by 9 cents — tighten the string" — so nothing depends on colour alone. ±5 cents counts as in tune.

7.2 Measured accuracy

The tuner was verified end to end in a real browser by feeding the application a synthesised guitar note as its microphone input, so everything downstream of the operating system's audio driver is the application's real code.

Fed in	Detected	Named	Verdict shown
82.50 Hz	82.50 Hz	E2, +2.0 cents	✓ In tune — E2
110.00 Hz	110.00 Hz	A2, +0.0 cents	✓ In tune — A2
147.00 Hz	147.00 Hz	D3, +2.0 cents	✓ In tune — D3
196.00 Hz	196.00 Hz	G3, +0.0 cents	✓ In tune — G3
247.00 Hz	247.00 Hz	B3, +0.4 cents	✓ In tune — B3
329.50 Hz	329.50 Hz	E4, −0.7 cents	✓ In tune — E4
41.00 Hz (bass low E)	41.00 Hz	E1, −8.6 cents	▲ Flat by 9 cents — tighten the string
55.00 Hz	55.00 Hz	A1, +0.0 cents	✓ In tune — A1
73.50 Hz	73.50 Hz	D2, +2.0 cents	✓ In tune — D2
98.00 Hz	98.00 Hz	G2, +0.0 cents	✓ In tune — G2
194.00 Hz (G3, flat)	194.00 Hz	G3, −17.7 cents	▲ Flat by 18 cents — tighten the string
250.50 Hz (B3, sharp)	250.50 Hz	B3, +24.8 cents	▼ Sharp by 25 cents — loosen the string
73.50 Hz, in Drop D	73.50 Hz	D2, +2.0 cents	✓ In tune — D2, targeting D2 rather than E2
110.00 Hz at 1/45th the level	110.00 Hz	A2, +0.1 cents	✓ In tune — A2
110.00 Hz at 1/120th the level	110.00 Hz	A2, +0.1 cents	✓ In tune — A2
41.20 Hz at 1/45th the level	41.00 Hz	E1, −8.6 cents	▲ Flat by 9 cents
147.00 Hz over a 2% DC offset	147.00 Hz	D3, +2.0 cents	✓ In tune — D3

The last four rows are the revision 6 work. The three quiet ones are the same tone at a fraction of the amplitude the others use, and they are read **just as accurately**, to a tenth of a cent — quiet does not mean approximate. The DC row is the case that used to give a reading three octaves out.

The detected frequency equals the input in every case, to the two decimal places the readout shows. The deviations in the table are the application correctly reporting how far the *input* sits from the target: 41.00 Hz really is 8.6 cents below E1, and the last row confirms the targets follow the tuning rather than assuming standard.

How quiet is quiet enough

Measured by feeding whole synthesised plucks, from the same model the application plays through, at falling amplitudes over a microphone noise floor:

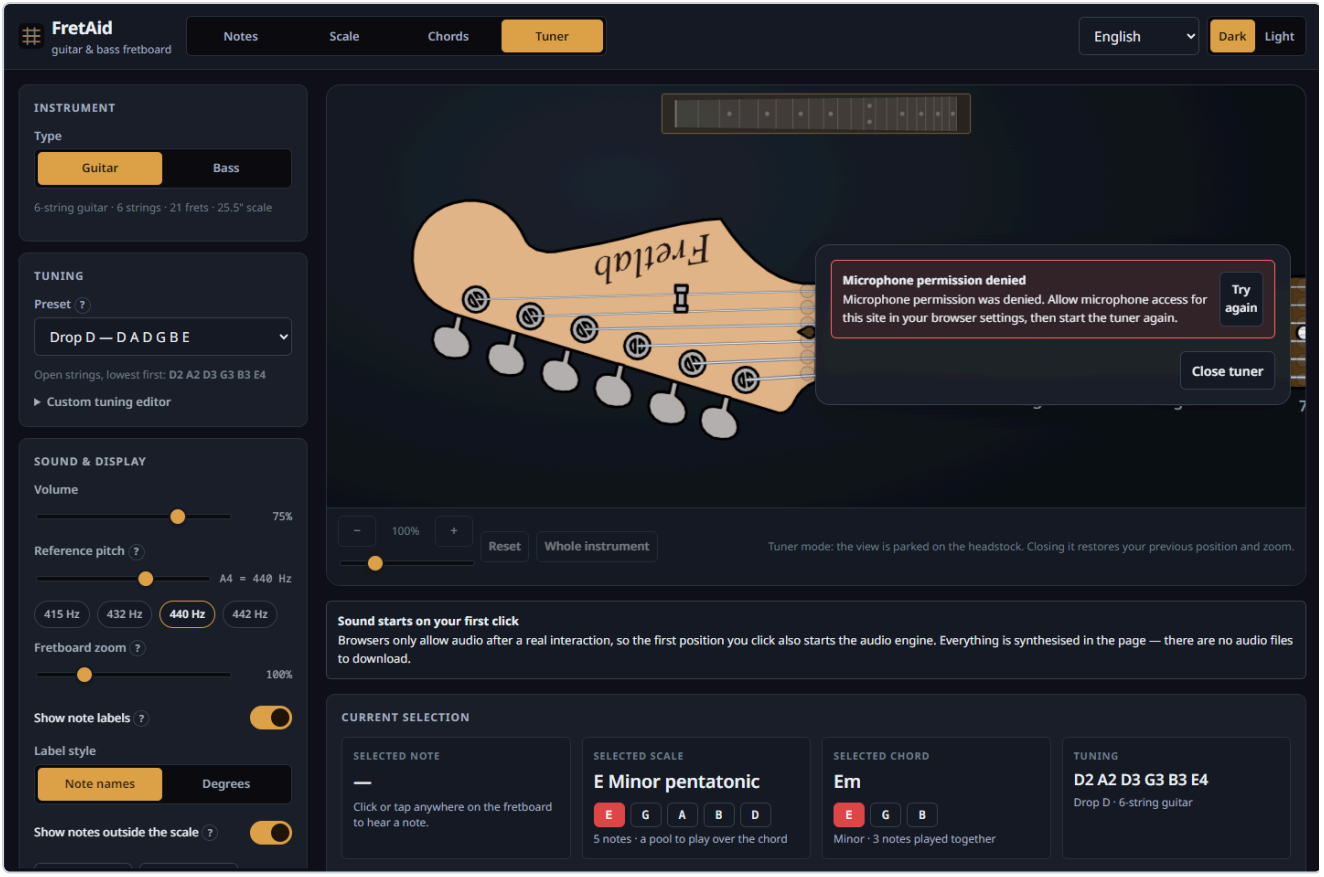
Pluck level	Before	After
Full strength	detected	detected
1/10th (−40 dBFS)	detected	detected
1/33rd (−50 dBFS)	**silence**	detected, clarity 0.97
1/100th (−57 dBFS)	**silence**	detected, clarity 0.97
1/200th (−62 dBFS)	**silence**	detected, clarity 0.97
1/500th (−70 dBFS)	**silence**	detected, clarity 0.87–0.95
1/1000th (−75 dBFS)	silence	silence

The pitch is correct to within a cent at every level that is detected at all, on the guitar's lowest and highest strings and the bass's. The improvement is **a factor of 50, about 34 dB** — and none of it came from making the detector cleverer. The detector was always able to read these signals; it was never being given them.

7.3 The camera move

Entering tuner mode does not open a popup somewhere on the screen. The current zoom level and horizontal scroll position are **stored**, then one CSS transform translates and scales the entire instrument so the headstock ends up enlarged and framed, with the tuner readout beside it and the tuning pegs lighting up as a string comes into tune. The stage itself grows to make room. The scale is clamped by both the width and the height of the view, so the headstock is never cropped.

Leaving tuner mode animates back along the same path and then restores the **stored** zoom and scroll position — not an arbitrary default. The transition honours `prefers-reduced-motion`.



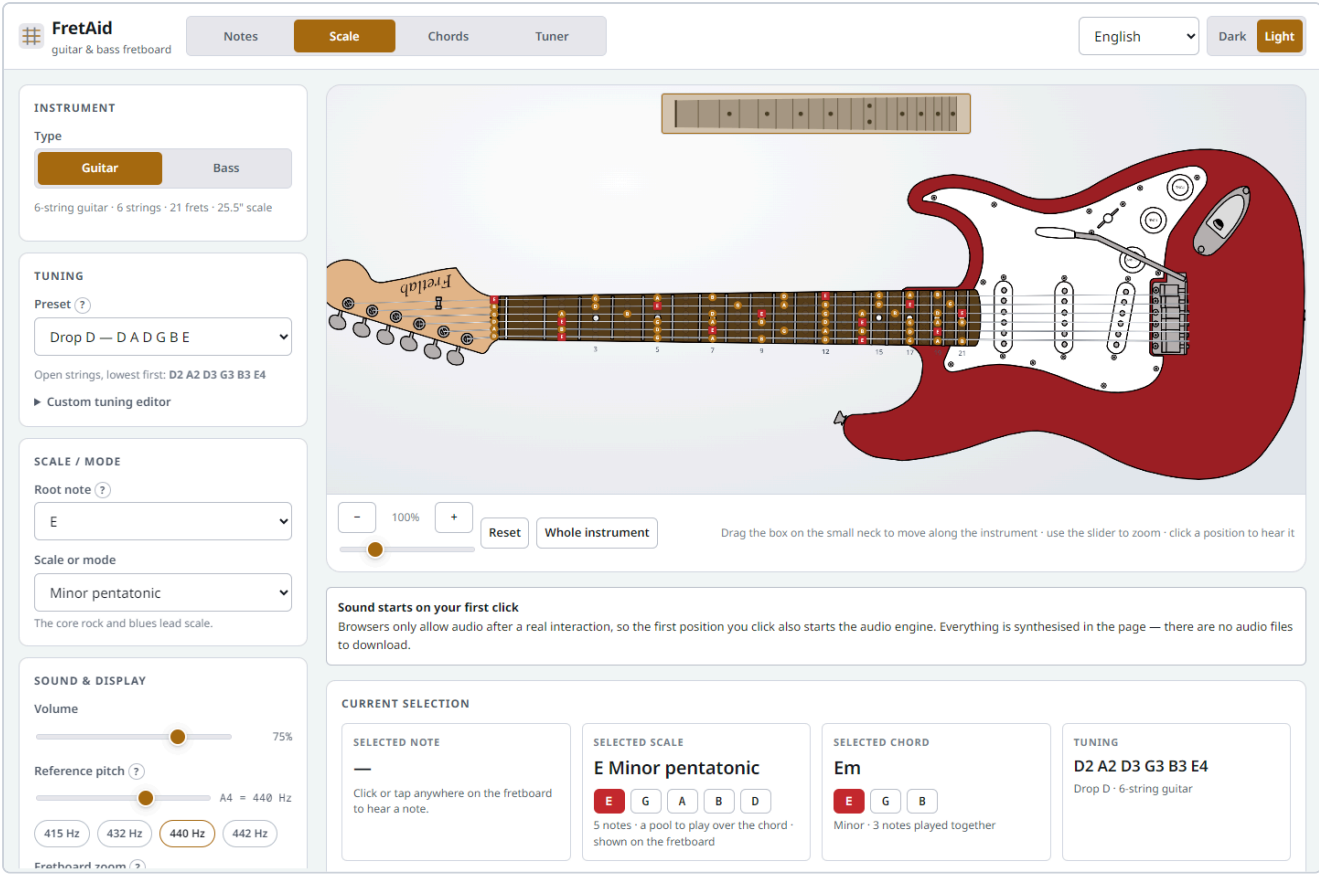
Tuner mode: the camera has travelled to the headstock, with the readout beside it. Shown here with microphone permission denied, which is handled with a plain-language explanation and a retry.

---

8. Interface, accessibility and resilience

8.1 Visual design

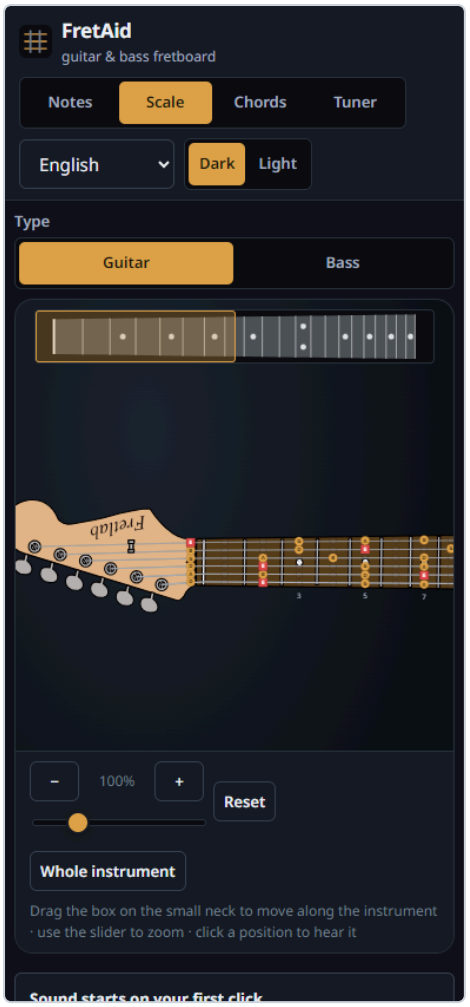
Noto Sans throughout (verified as actually loading, with a system-font fallback stack for when Google Fonts is unreachable), a restrained dark and light theme, and the fretboard kept as the clear visual focus. Musical roles are distinguished by **shape as well as colour** — roots are squares, scale notes are circles, chord-shape notes carry a halo, muted strings carry an × at the nut — so the display survives greyscale and colour-blindness.



The light theme.

8.2 Responsive design

The layout moves to a single column below 1080 px, with the instrument placed first because it is the point of the page and the guitar / bass switch lifted above it. The neck is moved by dragging the box on the map, pinch-to-zoom works, and controls keep a 42 px minimum touch target.



On a phone: the guitar / bass switch and the instrument come first, and the map above the neck moves the view.

8.3 Accessibility

- The fretboard is a keyboard-navigable grid: arrow keys move, Home and End jump to the nut and the last fret, Enter or Space plays, and the current position is announced.
- Every control has a real accessible name. One bug found this way during development — a toggle switch whose `aria-labelledby` pointed at an id containing spaces, so it never resolved and the switch was effectively unnamed — was caught by a test asserting that every control is named, and fixed.
- The tuner meter exposes its reading through `role="meter"` and live regions.
- Visible focus rings, a skip link, help text on unfamiliar controls, and text contrast meeting WCAG AA in both themes.

8.4 Error handling

Every failure path produces a clear message rather than a broken page: Web Audio missing, audio initialisation failing, microphone permission denied, no microphone present, an insecure (non-HTTPS) context, an unsupported browser, an invalid custom tuning, a chord with no playable voicing in the current tuning, and `localStorage` being blocked. A React error boundary catches any

remaining rendering failure and offers to clear the saved settings, which is the most likely way a stored value could break a future version.

8.5 Settings that persist

Instrument, tuning (including custom tunings per instrument), scale and root, chord and root, volume, zoom, label preferences, theme, strum direction and speed — all in `localStorage`, with no account and no backend. Tuner mode is deliberately *not* persisted, so reloading never reopens the microphone.

8.6 Six languages

The interface is offered in **English, 한국어, 日本語, 简体中文, 繁體中文 and Español**. The picker is in the header; the choice is saved with every other setting, and on a first visit the browser's own language preference is used.



The interface in Korean, with the scale catalogue translated as well as the controls.

One catalogue per language, and the compiler checks it. English is the source of truth: `src/i18n/en.ts` exports an object, and `Messages` is derived *from that object* with a mapped type. Every other language is declared as `Messages`, so a missing key, a misspelled key or a key left over from a deleted feature is a build error rather than a blank label discovered by a reader.

Sentences that embed a value are functions, not templates with holes. A message like "2 frets along, string 6" is written as

```
'fretboard.cell': ({ note, string, where }) => `${note}, ${string}번 현, ${where}`,
```

so each language puts the number, the note and the string where its own grammar wants them. Japanese counts frets with its own counter word, Chinese puts the string number before the word for string, and Korean marks it with 번. None of that is expressible by substituting into a fixed English word order, which is the usual reason translated interfaces read badly.

The music catalogues are translated, not transliterated. All 27 scales, all 21 chord types, the group headings and the "why this scale fits this chord" explanations have their own name and their own one-line description in each language — the longest part of the work, and the part that decides whether the application is actually usable by someone who does not read English. Note letters (A–G, ♯, ♭) are left as they are, because that is how they are written in every one of these languages.

Nothing is hard-coded that a reader can see. The tuning presets keep their note letters, which need no translation, and take their name from the catalogue: `Drop D — D A D G B E` becomes `드롭 D — D A D G B E`. The scale name shown in the information panel is composed in the selector that derives it, so the name in the footer, in the panel and in the chooser cannot disagree.

The cost. All six catalogues are in the main bundle, which is what took it from 236 kB to 393 kB (76 kB to 131 kB gzipped). They could be split and fetched on demand, but that would mean the first paint in Korean is in English until the catalogue arrives, and it would make the synchronous lookup the crash screen and the selectors rely on asynchronous. For a page this size the 55 kB was the better trade.

Scripts are loaded only when needed. Latin locales download nothing extra. Choosing Korean, Japanese or Chinese injects exactly one Google Fonts link for that script and sets `<html lang>`, which is also what tells the browser which Han glyph variants to draw — the reason Simplified and Traditional Chinese are separate locales rather than one "Chinese" with converted text.

What the tests check. Beyond the type-level guarantee: that no value is blank in any language; that every message English renders through a function is still a function elsewhere (a translation that flattened one into a fixed string would silently drop a number from the interface); that every such message actually interpolates its arguments; that nothing in the four CJK catalogues is still ASCII prose left over from a copy of the English; that every scale, chord, category and tuning id is covered in every language; that `zh-TW`, `zh-HK` and `zh-MO` detect as Traditional while `zh-CN`, `zh-SG` and a bare `zh` detect as Simplified; and, driving the real application, that switching the picker retranslates the panels, the catalogue and all 132 fretboard position names, sets the document language, and survives a reload.

---

9. Testing results

`npm test` — **280 tests, 9 files, all passing** in about a minute.

Area	Tests	What is checked
Pitch	14	name ↔ MIDI round trip over the whole range; frequencies against published values; cents; nearest-note boundaries; malformed names rejected
Spelling	10	B♭ major; F♯ major with its E♯; modal spellings; the blues ♭5/♮5 on one letter; C°7's double flat; key-aware fretboard maps
Fretboard	22	every tuning preset of both instruments; Drop D changing only one string; D standard shifting all six; custom tunings; highlighting moving with the tuning; tuning validation
Scales	15	relative modes sharing one pitch-class set; harmonic vs melodic minor; pentatonics; blues; symmetric scales; degree labels
Chords	34	every chord type's pitch classes; the solver finding real open shapes; **every returned shape verified to sound only the chord it claims**; shapes recalculated per tuning; power chords across seven instrument/tuning combinations
Audio	13	rendered notes in tune within 5 cents across both instruments' ranges and three sample rates; harmonic content; decay; determinism
Tuner	34	a string played softly, down to 1/500th of full strength, and still to a cent; a DC-offset microphone; room noise and white noise still rejected at every level; the gate following the room, warming up, not being raised by a note, and its ceiling; the decibel signal meter; every open string of every standard tuning; a 31 Hz low B; no octave errors; silence and noise rejected; cents; targets following the current tuning

Area	Tests	What is checked
Translations	75	every language covering every interface key, scale, chord, category and tuning; no blank values; parameterised messages still parameterised and still substituting; no English left in the CJK catalogues; browser-language detection, including the Chinese variants
Application	63	the stage keeping its height when zoomed, with the neck centred; settings saved under the old name still read; the map showing the neck alone with all 21 frets, and moving the view by drag, press and keyboard; the wheel not zooming; the zoom slider; the guitar / bass switch moving to the top on a narrow screen; both instruments; tuning changes recalculating the rendered neck; the custom-tuning editor; chord shapes per tuning; the Drop D one-finger power chord; down vs up strum ordering; muted strings silent; strum speed; microphone requested only in tuner mode and released on exit; view saved and restored; persistence; keyboard navigation

The application tests render the real application in jsdom and drive it the way a visitor would, rather than testing a stub.

9.1 Checks run in a real browser

Three scripts in `tools/` drive the built application in Chrome. They confirmed:

- **No console errors or warnings** in any mode, instrument, theme or viewport.
- **Noto Sans genuinely loading** (`document.fonts.check` returns true).
- **Real audio**: clicking a fret produces a 3.3-second buffer with a peak amplitude of 0.92 — actual audio, not silence.
- **Strum direction**: a down strum schedules six sources at an even 32 ms apart; an up strum schedules the identical set in exactly reversed order.
- **The tuner camera** transform applying on entry and clearing on exit.
- **The tuner readings** in section 7.2.

9.2 Items from the brief's development checklist

Check	Result
Build the application	Passes; `tsc --noEmit` clean
Run locally	`npm run dev`, verified
Fix build and console errors	No build errors, no console errors
Tuning changes recalculate fret notes	Verified by unit and UI tests
Both instrument configurations	Verified, each separately
Bass separately	Verified, including Drop D and its 41 Hz low E
Every language	Verified: 75 translation tests, and the interface driven in Korean
Scale highlighting	Verified
Note playback	Verified in a real browser
Chord playback	Verified in a real browser
Normal / down / up strumming	Verified, with measured timing
Tuner mode and its animation	Verified, with measured transforms
Exiting the tuner restores the previous view	Verified by test and in the browser
Mobile responsiveness	Verified at 390 × 844
Noto Sans actually in use	Verified in the browser
Production build works	Verified via `npm run preview`

---

10. Running and deploying

10.1 Locally

```
npm install
npm run dev        # http://localhost:5173
npm test
npm run build      # type-checks, then writes dist/
npm run preview    # serves dist/ at http://localhost:4173
```

Node 18 or newer; 20 LTS recommended. `http://localhost` counts as a secure context, so the microphone tuner works in development with no certificate setup.

10.2 Free hosting, no domain required

The build sets `base: './'`, so the same `dist/` folder works from a site root *or* from a subdirectory. Nothing needs reconfiguring per host.

GitHub Pages is the recommended option and is already wired up. `.github/workflows/deploy.yml` installs, runs the tests, builds and publishes on every push to `main`. The only manual step is choosing **GitHub Actions** under Settings → Pages. The site then appears at `https://<username>.github.io/<repository>/`, over HTTPS, which is what the microphone tuner needs.

`netlify.toml` and `vercel.json` are included for those platforms, and Cloudflare Pages needs only `npm run build` with an output directory of `dist`. Netlify also accepts the `dist/` folder by drag and drop with no build step at all.

10.3 Microphone permission

Permission is requested only on entering tuner mode — loading the page never prompts. Audio is analysed in the page and discarded frame by frame; nothing is recorded or transmitted. Leaving tuner mode stops the media tracks, so the browser's recording indicator goes out. The README documents how to re-enable a denied microphone in each major browser.

---

11. What is supported

11.1 Instruments

Instrument	Strings	Frets	Default tuning	Presets
6-string guitar	6	21	E2 A2 D3 G3 B3 E4	13 + custom
4-string bass	4	21	E1 A1 D2 G2	7 + custom

The 7-string guitar and 5-string bass were removed in revision 3, at the client's request.

11.2 Tuning presets

6-string guitar — Standard (E A D G B E), Half step down, D standard, C♯ standard, Drop D, Drop C♯, Drop C, Drop B, Drop A, Open G, Open D, Open E, DADGAD, plus custom.

4-string bass — Standard (E A D G), Half step down, D standard, Drop D, Drop C, BEAD, Tenor, plus custom.

Every preset name is translated; the open-string letters are not, because they are written the same way in all six languages.

11.3 Scales and modes (27)

- **Major / minor** — Major (Ionian), Natural minor (Aeolian), Harmonic minor, Melodic minor, Harmonic major
- **Modes** — Ionian, Dorian, Phrygian, Lydian, Mixolydian, Aeolian, Locrian, Lydian dominant, Phrygian dominant, Altered
- **Pentatonic** — Major pentatonic, Minor pentatonic, Hirajoshi
- **Blues** — Blues (minor), Blues (major)
- **Symmetric** — Whole tone, Diminished (half–whole), Diminished (whole–half), Chromatic
- **Exotic** — Hungarian minor, Double harmonic (Byzantine), Bebop dominant

11.4 Chords (21)

- **Triads** — Major, Minor, Diminished, Augmented, Major 6th, Minor 6th
- **Sevenths** — Dominant 7th, Major 7th, Minor 7th, Minor 7♭5, Diminished 7th, Minor major 7th, 7sus4
- **Suspended** — Sus2, Sus4
- **Power** — root + 5th, root + 5th + octave
- **Extended** — add9, Dominant 9th, Minor 9th, Major 9th

Each is available on all 17 root spellings (C, C♯, D♭, D, … B), with voicings calculated for whatever tuning is active.

11.5 Languages

Language	Picker reads	Document language
English	English	`en`
Korean	한국어	`ko`
Japanese	日本語	`ja`
Simplified Chinese	简体中文	`zh-Hans`
Traditional Chinese	繁體中文	`zh-Hant`
Spanish	Español	`es`

Each covers the complete interface and the complete scale, chord and tuning catalogues.

---

12. Honest limitations

A few things are worth stating plainly rather than leaving to be discovered.

- **Voicings are limited to a four-fret span and four fingers.** That is the right default for a learning tool, but it means some wide jazz voicings and thumb-over shapes are not offered.
- **The body is drawn to a compressed scale.** A real electric guitar's body is about six times the width of its neck. Drawn to that ratio, with the neck still large enough to tap accurately, the instrument would need roughly twice the vertical space the page can give it. The body is therefore about half the size relative to the neck that it should be, which makes the neck look chunky next to it. The silhouette is correct; the proportion is a deliberate trade in favour of a usable fretboard, and it is the one place where the drawing is knowingly not to scale.
- **The tuner is monophonic**, like every tuner of this kind. It expects one string at a time, and says so.
- **Microphone capture could not be tested against real hardware here.** The signal path was verified instead by feeding the application a synthesised note as a genuine `MediaStream`, which exercises everything except the operating system's audio driver.
- **The translations have not been reviewed by a native speaker.** They are mine. The interface strings are straightforward; the music terminology is where a reviewer would earn their keep, since several terms have both a borrowed and a native form in Korean and Japanese and the choice between them is a question of what learners in each country actually say. Each language is one file, so a correction is a one-line change.
- **Playback is synthesis, not a recording.** It is a convincing plucked string rather than a studio-recorded guitar, which is the deliberate trade for having no audio files and no licensing questions.

12.1 Natural next steps

Adding an 8-string guitar, a 6-string bass or a ukulele is a data change only, and so is a seventh language: copy `src/i18n/en.ts` and its music catalogue, translate the values, and the compiler names anything missed. Beyond that, the obvious additions would be chord progressions with playback, a metronome, saving custom scales, and exporting a chord diagram as an image.

---

13. Deliverables

Source	`C:\Users\hahah\fretlab`
Documentation	`README.md` — install, run, build, deploy, microphone, browser support, architecture, engine explanations, full catalogues, test results
This report	`docs/report.md`, `docs/FretAid-report.docx`, `docs/FretAid-report.pdf`
Deployment	`.github/workflows/deploy.yml`, `netlify.toml`, `vercel.json`
Languages	`src/i18n/` — one file per language, plus the English source of truth
Browser check scripts	`tools/screenshots.mjs`, `tools/audio-check.mjs`, `tools/tuner-check.mjs`, `tools/catalogue.mjs`
Screenshots	`docs/images/`

The project can be run immediately with `npm install && npm run dev`, and deployed to GitHub Pages by pushing it to a repository and selecting GitHub Actions as the Pages source. No domain purchase is required at any point.